CI/CD AUTOMATIZACIJA DOCKER PROJEKTA

Ovaj vodic opisuje postupak od zavrsenog Docker projekta do automatskog build-a, objavljivanja image-a na Docker Hub i deployment-a na virtuelnu masinu.

1. CILJ AUTOMATIZACIJE

Tok procesa:

```
Izmena koda
   |
   v
git push origin main
   |
   v
GitHub Actions: build image-a
   |
   v
Docker Hub: novi image-i
   |
   v
Self-hosted runner na VM-u
   |
   v
docker compose pull i up -d
   |
   v
Nova verzija aplikacije na VM-u
```

Automatizacija je uspesna kada posle obicnog push-a na main granu GitHub sam izgradi image-e, posalje ih na Docker Hub i osvezi aplikaciju na VM-u.

2. PRIPREMA DOCKER PROJEKTA

Projekat treba da ima:

- Dockerfile za aplikacioni servis, na primer PHP-FPM;
- Dockerfile za web servis, na primer Nginx;
- Compose fajl za development;
- Compose fajl za deployment;
- Git repozitorijum na GitHub-u;
- Docker Hub repozitorijume za image-e.

Lokalno prvo proveriti development okruzenje:

```
docker compose -f compose.dev.yaml up -d --build
docker compose -f compose.dev.yaml ps
```

Zaustavljanje development okruzenja:

```
docker compose -f compose.dev.yaml down --remove-orphans
```

Deployment Compose fajl treba da koristi gotove image-e sa Docker Hub-a:

```
services:
  web:
    image: sinisaglavas/docker-project-web:latest

  php-fpm:
    image: sinisaglavas/docker-project-php-fpm:latest
```

Development fajl moze koristiti build, dok deploy fajl treba da koristi image. Tako VM ne mora da build-uje aplikaciju.

3. DOCKER HUB

Kreirati repozitorijume:

```
sinisaglavas/docker-project-php-fpm
sinisaglavas/docker-project-web
```

Kreirati Docker Hub Access Token sa pravom za push image-a. Token se ne upisuje direktno u workflow

i ne salje se u Git repozitorijum.

4. GITHUB SECRETS

U GitHub repozitorijumu otvoriti:

Settings -> Secrets and variables -> Actions

Dodati:

DOCKERHUB_USERNAME
DOCKERHUB_TOKEN

DOCKERHUB_TOKEN treba da bude Docker Hub Access Token, a ne obicna lozinka.

Produkcioni .env ne treba slati na GitHub. On ostaje na VM-u i mora biti u .gitignore fajlu.

5. GITHUB ACTIONS WORKFLOW

Workflow se cuva u:

.github/workflows/deploy.yml

Primer kompletnog workflow-a:

```
name: CI/CD Pipeline

on:
  push:
    branches:
      - main
  workflow_dispatch:

jobs:
  build-and-push:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and Push PHP-FPM Image
        uses: docker/build-push-action@v5
        with:
          context: .
          file: ./docker/common/php-fpm/Dockerfile
          target: production
          push: true
          tags: sinisaglavas/docker-project-php-fpm:latest

      - name: Build and Push Nginx Image
        uses: docker/build-push-action@v5
        with:
          context: .
          file: ./docker/production/nginx/Dockerfile
          push: true
          tags: sinisaglavas/docker-project-web:latest

  deploy:
    needs: build-and-push
    runs-on: self-hosted

    steps:
      - name: Deploy application on VM
        run: |
          cd /home/devops/docker_project
```

```
docker compose -f compose.deploy.yaml pull
docker compose -f compose.deploy.yaml up -d --remove-orphans
docker compose -f compose.deploy.yaml ps
```

workflow_dispatch omogucava rucno pokretanje preko dugmeta Run workflow. Push na main omogucava potpuno automatsko pokretanje. needs: build-and-push obezbeuje da deployment pocne tek kada su image-i uspesno izgraeni i poslati na Docker Hub.

6. PRIPREMA VM-A

Proveriti Docker i Compose:

```
docker --version
docker compose version
sudo systemctl status docker
```

Projekat treba da postoji na istoj putanji koju koristi workflow:

```
cd /home/devops/docker_project
git status
ls -la .env compose.deploy.yaml
```

Na VM-u mora postojati produkcioni .env. Taj fajl ne treba slati na GitHub.

Prvi rucni deployment:

```
docker compose -f compose.deploy.yaml pull
docker compose -f compose.deploy.yaml up -d
docker compose -f compose.deploy.yaml ps
```

Pre automatizacije potvrditi da aplikacija radi preko IP adrese VM-a.

7. INSTALACIJA SELF-HOSTED RUNNER-A

Na GitHub-u otvoriti:

```
Repository -> Settings -> Actions -> Runners -> New self-hosted runner
```

Izabrati Linux -> x64. GitHub prikazuje komande za preuzimanje i konfiguraciju runner-a. Te komande treba kopirati iz GitHub-a, jer se verzija runner-a menja.

Na VM-u koristiti poseban folder:

```
mkdir -p ~/actions-runner
cd ~/actions-runner
```

Posle raspakivanja pokrenuti GitHub-ovu konfiguracionu komandu:

```
./config.sh --url https://github.com/korisnik/repozitorijum --token NOVI_TOKEN
```

Token je privremen i ne treba ga deliti ili objavljivati.

Testiranje:

```
./run.sh
```

Ocekivane poruke:

```
Connected to GitHub
Listening for Jobs
```

8. POKRETANJE RUNNER-A KAO SERVISA

Rucno pokretanje sa ./run.sh radi samo dok je terminal otvoren.

Za automatski rad posle restarta VM-a, iz foldera u kome se nalazi run.sh pokrenuti:

```
sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status
```

Provera servisa:

```
systemctl list-units 'actions.runner.*'
```

Ako je status active (running), runner ce automatski raditi posle restarta VM-a.

Runner korisnik mora imati pristup Docker-u. Ako se pojavi permission denied:

```
sudo usermod -aG docker $USER
```

Posle toga odjaviti se i ponovo prijaviti ili restartovati VM.

9. PRVI TEST AUTOMATIZACIJE

Proveriti da je runner na GitHub-u Active ili Idle, zatim promeniti neki fajl i izvrsiti:

```
git status
git add IZMENJENI_FAJL
git commit -m "test: automatic deployment"
git push origin main
```

Na GitHub-u otvoriti Actions -> CI/CD Pipeline.

Ocekivani job-ovi:

```
build-and-push: success
deploy: success
```

Ako prvi job uspe, image-i su na Docker Hub-u. Ako drugi uspe, VM je preuzeo image-e i pokrenuo novu verziju.

10. PROVERA DEPLOYMENT-A NA VM-U

```
cd /home/devops/docker_project
docker compose -f compose.deploy.yaml ps
```

Logovi:

```
docker compose -f compose.deploy.yaml logs --tail=100
docker compose -f compose.deploy.yaml logs --tail=100 web
docker compose -f compose.deploy.yaml logs --tail=100 php-fpm
docker compose -f compose.deploy.yaml logs --tail=100 mysql
```

Provera image-a:

```
docker images | grep docker-project
```

Aplikaciju proveriti preko IP adrese VM-a.

11. NAJCESCI PROBLEMI

YAML greska

Proveriti da su name, on i jobs na pocetku reda i da se koriste razmaci, ne tabulatori.

deploy je Queued

Proveriti da runner radi i da postoji:

```
Listening for Jobs
```

U workflow-u mora pisati:

```
runs-on: self-hosted
```

Git pull odbija deployment

Deployment workflow ne mora da radi git pull ako compose.deploy.yaml vec postoji na VM-u i kod je ugraen u Docker image. U tom slucaju koristiti samo:

```
docker compose -f compose.deploy.yaml pull
docker compose -f compose.deploy.yaml up -d --remove-orphans
```

CI/CD je uspesan, ali se promena ne vidi u aplikaciji

Ovo se cesto desava kada GitHub Actions objavljuje image sa tagom latest, a compose.deploy.yaml na VM-u koristi stariji tag, na primer 2.0.

Provera image tagova u Compose fajlu:

```
cd /home/devops/docker_project
grep -n "image:" compose.deploy.yaml
```

Komanda grep -n prikazuje linije koje sadrze image i broj linije na kojoj se nalaze. Rezultat treba da koristi isti tag koji koristi GitHub Actions. Ako workflow objavljuje latest, u Compose fajlu treba da bude:

```
image: sinisaglavas/docker-project-web:latest
image: sinisaglavas/docker-project-php-fpm:latest
```

Ako Compose fajl koristi 2.0, a workflow objavljuje latest, promeniti tagove:

```
sed -i 's/:2\.0/:latest/g' compose.deploy.yaml
```

Komanda sed -i menja fajl direktno. Deo s/:2\.0/:latest/g znaci da se svaka pojava taga :2.0 zameni tagom :latest.

Proveriti da je izmena uspesna:

```
grep -n "image:" compose.deploy.yaml
```

Preuzeti najnovije image-e sa Docker Hub-a:

```
docker compose -f compose.deploy.yaml pull
```

Komanda pull preuzima image-e navedene u Compose fajlu. Bez ove komande VM moze nastaviti da koristi staru lokalnu verziju image-a.

Ponovo kreirati kontejnere:

```
docker compose -f compose.deploy.yaml up -d --force-recreate --remove-orphans
```

Opcija -d pokrece kontejnere u pozadini. Opcija --force-recreate obavezno kreira nove kontejnere, cak i kada Compose misli da promena nije potrebna. Opcija --remove-orphans uklanja kontejnere koji vise nisu definisani u trenutnom Compose fajlu.

Proveriti koji image-i se stvarno koriste:

```
docker compose -f compose.deploy.yaml images
```

Ova komanda prikazuje image-e koje koriste aktivni Compose servisi. Treba proveriti da li je tag latest, a ne stari tag 2.0.

Ocistiti Laravel cache nakon promene Blade fajla:

```
docker compose -f compose.deploy.yaml exec -T php-fpm php artisan optimize:clear
docker compose -f compose.deploy.yaml exec -T php-fpm php artisan view:clear
```

Prva komanda cisti Laravel konfiguracioni, route, event i application cache. Druga brise kompajlirane Blade view fajlove. Opcija -T iskljucuje pseudo-terminal, sto je potrebno kada se komanda izvrsava iz GitHub Actions workflow-a.

Proveriti da li je novi fajl zaista u aktivnom kontejneru:

```
docker compose -f compose.deploy.yaml exec -T php-fpm php -r 'echo
strpos(file_get_contents("resources/views/welcome.blade.php"), "NOVI_TEKST") !== false ? "FOUND\n"
: "NOT FOUND\n";'
```

Zameniti NOVI_TEKST delom teksta koji je dodat u welcome.blade.php. Rezultat FOUND znaci da aktivni kontejner sadrzi novu verziju fajla. Rezultat NOT FOUND znaci da je pokrenut stari image ili da izmena nije bila deo GitHub commit-a.

Ako komanda exec prijavi da php-fpm nije pokrenut, proveriti sve kontejnere:

```
docker compose -f compose.deploy.yaml ps -a
```

Status Healthy ili Up znaci da servis radi. Status Exited (0) za migrate je normalan, jer je migrate jednokratni servis koji se zaustavlja posle uspesnih migracija. Ako je php-fpm zaustavljen, pogledati logove:

```
docker compose -f compose.deploy.yaml logs --tail=100 php-fpm
docker compose -f compose.deploy.yaml logs --tail=100 migrate
```

Na kraju uraditi potpuno osvezavanje browsera:

```
Ctrl + F5
```

Za trajno resenje, tagovi u workflow-u i compose.deploy.yaml moraju uvek biti usklaeni. Ako workflow objavljuje latest, deployment Compose fajl treba da koristi latest. Izmenu compose.deploy.yaml treba commit-ovati na GitHub da bi konfiguracija ostala dokumentovana i dostupna na svim masinama.

No space left on device

Provera:

```
df -h
docker system df
```

Bezbednije ciscenje:

```
docker container prune
docker image prune -a
sudo apt clean
```

Ne koristiti rutinski:

```
docker system prune -a --volumes
```

jer moze obrisati volume-e i podatke baze.

Push je odbijen jer je remote ispred lokalne grane

Sacuvati lokalne izmene, preuzeti remote izmene, pa vratiti stash:

```
git stash push -u -m "local changes before syncing main"
git pull --ff-only origin main
git stash pop
```

Ako se pojavi konflikt:

```
git status
git diff --name-only --diff-filter=U
```

Ne koristiti git push --force bez jasnog razloga.

12. BEZBEDNOSNE PREPORUKE

- .env, lozinke i tokeni ne smeju biti u Git repozitorijumu.
- Docker Hub token koristiti kroz GitHub Secrets.
- Runner token ne deliti u screenshot-u ili poruci.
- Ne koristiti docker compose down -v bez provere, jer moze obrisati bazu.
- Ne koristiti git reset --hard ako izmene nisu sacuvane.
- Za ozbiljniji projekat koristiti immutable tag, na primer Git commit SHA, umesto samo latest.
- Redovno proveravati slobodan prostor VM-a.

13. KONACNA KONTROLNA LISTA

Automatizacija je zavrsena kada:

- projekat postoji na GitHub-u;
- Dockerfile-i uspesno grade image-e;
- image-i postoje na Docker Hub-u;
- GitHub Secrets su podeseni;
- workflow postoji u .github/workflows/deploy.yml;
- workflow ima build-and-push job;
- workflow ima deploy job sa runs-on: self-hosted;
- VM ima Docker i Compose;

- VM ima .env i compose.deploy.yaml;
- self-hosted runner je registrovan;
- runner radi kao systemd servis;
- build-and-push i deploy su zeleni na GitHub-u;
- aplikacija je dostupna preko IP adrese VM-a posle git push origin main.